Sentinel-1 SAR Oil Spill Predictor

Viva / Study Guide — Terms, Values, and How They Work in Your Project

Rule: Every term below is limited to a maximum of three sentences: what it means, how it relates to your project, and—where supported—why the value/choice is used.

Source boundary: This guide is based on your uploaded technical documentation; when the document does not state why a particular value was chosen, that is explicitly noted rather than guessed.

A. Input & Data

1. Sentinel-1

A satellite mission that provides radar imagery. Your project uses its imagery as the source data for oil-spill segmentation.

2. SAR (Synthetic Aperture Radar)

Radar imaging that measures returned electromagnetic signals rather than ordinary visible-light photographs. Your model learns patterns in SAR backscatter to identify oil-spill-related regions.

3. VV / VH Polarization

VV means vertical transmit/vertical receive, while VH means vertical transmit/horizontal receive. Your model uses these two channels as its input instead of three RGB channels.

4. TIFF (.tif)

A raster image format that can store numerical pixel data and geospatial information. Your pipeline reads the Sentinel-1 TIFF files and later preserves their geographic reference information.

5. Raster

A raster represents information as a grid of pixels/cells, with values stored at each location. Your SAR images and segmentation masks are processed as raster grids.

6. Pixel

A single cell in an image grid containing numerical information. In your segmentation task, each pixel ultimately receives a predicted class.

7. Dataset

A collection of examples used by the machine-learning system. Your examples contain SAR imagery and corresponding ground-truth masks.

8. Ground Truth / Mask

The correct labeled map used as the reference answer during training and evaluation. Your model compares its predicted segmentation with these masks.

9. Train / Validation / Test Split

Data is divided so the model can learn, be monitored, and finally be tested on unseen data. Your documentation specifies 70% training, 15% validation, and 15% held-out test data.

10. Data Leakage

Data leakage occurs when information that should remain unseen influences training or model selection. Your documentation says the 180 test images were isolated before training to prevent this.

11. Data Augmentation

Training images are transformed, such as by rotations and flips, to provide more variation. In your project augmentation was applied only to the training pool.

B. Machine Learning Fundamentals

12. Machine Learning

A method where a computer learns patterns from examples instead of being given every rule explicitly. Your model learns the relationship between SAR imagery and labeled segmentation masks.

13. Deep Learning

A branch of machine learning that uses neural networks with many learned layers. Your project uses a deep neural network for pixel-wise image segmentation.

14. Neural Network

A mathematical model made of layers containing learnable parameters. In your project those parameters are adjusted during training so predictions become closer to the ground-truth masks.

15. Weight / Parameter

A numerical value inside the neural network that is learned during training. The training process changes these values so the model produces better segmentation predictions.

16. Feature

A pattern or representation that helps the model distinguish different regions. Early layers can learn simpler patterns while deeper layers combine them into more useful representations.

17. Supervised Learning

Learning from examples where the correct answer is provided. Your images have corresponding masks, so the model can compare its prediction with the known answer.

18. Classification

Assigning an item to a category. Your project performs classification at the pixel level rather than giving only one class to the entire image.

19. Semantic Segmentation

Assigning a semantic class to each pixel in an image. This is the main machine-learning problem your U-Net solves.

20. Class Imbalance

A situation where some classes contain far more pixels than others. Your documentation notes that oil occupies a small fraction of ocean pixels, which is why ordinary accuracy can be misleading.

C. CNN / U-Net / ResNet

21. CNN (Convolutional Neural Network)

A neural network designed to learn spatial patterns in images using convolution operations. Your ResNet34 encoder uses convolutional processing to extract features from SAR data.

22. Convolution

A mathematical operation that applies a small learnable filter across nearby pixels to detect patterns. Your network uses convolutions to transform SAR pixels into increasingly useful feature maps.

23. Kernel / Filter

A small set of learnable numbers used by a convolution to detect a pattern. During training the network learns useful filter values rather than having them manually specified.

24. Feature Map

The output produced by applying learned filters to an input, showing where particular learned patterns appear. Feature maps move through the encoder and decoder as the image is processed.

25. U-Net

A segmentation architecture with an encoder and decoder connected by skip connections. Your U-Net converts the extracted features into a pixel-level prediction.

26. Encoder

The part of a network that extracts features and gradually builds more abstract representations. In your project ResNet34 acts as the U-Net encoder.

27. Decoder

The part that reconstructs a detailed spatial output from deeper feature representations. Your U-Net decoder uses encoder information to produce the final segmentation map.

28. ResNet34

A residual neural network with 34 layers used as the encoder/backbone. Your documentation says it was chosen for a balance of feature extraction capability and memory efficiency on the 8 GB GPU.

29. Residual Connection

A shortcut that adds the original input to a transformed output, commonly written as F(x) + x. ResNet uses these connections to improve information and gradient flow through the network.

30. Skip Connection

A connection that passes earlier high-resolution features directly to later decoder layers. In your U-Net this helps combine detailed spatial information with deeper semantic information.

31. Bottleneck

The compressed representation near the deepest part of the encoder-decoder network. Your documentation explains that U-Net uses skip connections because compression alone can lose precise spatial information.

32. Input Channels = 2

Each input sample has two channels rather than RGB's three. They correspond to Sentinel-1 VV and VH data.

33. Output Classes = 5

The model predicts five semantic categories for pixels. Your classes are Sea Surface, Oil Spill, Look-alike, Ship, and Land.

D. Training & Optimization

34. Training

Training is the repeated process of making predictions, measuring error, and changing network parameters. Your model performs this process over the training dataset.

35. Forward Pass

The input is sent through the network to produce a prediction. In your project the two-channel SAR tensor passes through ResNet34/U-Net to produce class predictions.

36. Loss Function

A numerical measure of how different the prediction is from the target. Training uses the loss to determine how the model should change.

37. Dice Loss

A loss based on overlap between predicted and ground-truth regions; Dice is $2|X \cap Y|/(|X|+|Y|)$, and Dice loss is 1 minus the differentiable Dice score. Your project uses it because the documentation identifies severe oil-versus-background class imbalance.

38. Gradient

A mathematical value describing how the loss changes when a model parameter changes. Gradients are used to decide how the network parameters should be updated.

39. Backpropagation

The process of calculating how the loss should be attributed backward through the network. It supplies gradients used during parameter updates.

40. Optimizer

An algorithm that uses gradients to update the model's learned parameters. Your supplied documentation does not identify the optimizer, so its specific choice should be checked in your training code.

41. Learning Rate

A parameter controlling how large each optimizer update is. The supplied documentation does not state its value, so it should be checked in your training code rather than guessed.

42. Epoch

One complete pass through the training dataset. Your documentation says the model was trained for 20 epochs, but it does not establish that 20 was mathematically optimal.

43. Batch Size

The number of samples processed together before an update. Your batch size was reduced to 2 and sometimes 1 because larger batches caused CUDA out-of-memory problems on the 8 GB GPU.

44. VRAM

The memory available on the GPU for model data and intermediate computations. Your RTX 4060 has 8 GB of physical VRAM, which constrained the batch size.

45. OOM (Out of Memory)

An error that occurs when the GPU cannot allocate enough memory for the requested computation. Reducing batch size and using mixed precision were used to address this in your project.

46. AMP (Automatic Mixed Precision)

A technique that uses lower numerical precision for suitable operations to reduce memory and computation requirements. Your documentation says torch.amp.autocast was used during training.

47. FP32 / FP16

FP32 uses 32-bit floating-point values while FP16 uses 16-bit values. Your project uses mixed precision so suitable calculations can use lower precision without forcing the entire computation to FP16.

48. Checkpoint

A saved copy of model parameters from training. Your documentation identifies best_oilspill_model.pth as the checkpoint used for final evaluation.

E. Evaluation

49. Validation Set

A portion of the data used during development to monitor how the model behaves on data not used for its parameter updates. Your project uses 15% validation data.

50. Test Set

Data kept unseen until final evaluation. Your documentation describes 180 locked test images used to measure the final model.

51. Overfitting

When a model becomes too specialized to training examples and performs worse on unseen data. Similar training and validation loss can be evidence worth examining, but your document's claim of 'zero overfitting' is stronger than that evidence alone proves.

52. IoU (Intersection over Union)

IoU measures overlap as intersection divided by union between prediction and ground truth. Your documentation reports 83.63% mean image-wise IoU and 65.71% global IoU for the oil class.

53. Dice Score

Dice measures overlap using twice the intersection divided by the total predicted and ground-truth sizes. Your documentation reports 80.42% mean Dice for the oil class.

54. Accuracy

The proportion of predictions that are correct. For heavily imbalanced segmentation, high accuracy can occur even when the small oil class is poorly detected, so overlap metrics are important.

F. Inference & Geospatial Output

55. Inference

Using a trained model to make predictions without updating its parameters. Your app_predict.py pipeline loads the trained model and predicts on TIFF imagery.

56. torch.no_grad()

A PyTorch mode that stops gradient tracking when gradients are not needed. Your inference pipeline uses it to avoid training-related gradient storage.

57. Normalization

Scaling numerical input values into a suitable range. Your documentation says the TIFF values are converted to float32 and normalized to 0–1.

58. Connected Component Analysis

A method for finding separate connected regions in a binary mask. Your project uses OpenCV connectedComponentsWithStats to identify small predicted regions that can be removed.

59. 50-Pixel Threshold

A post-processing rule that removes connected regions smaller than 50 pixels. Your documentation states the threshold but does not explain how it was experimentally selected.

60. Noise / False Positive

A predicted region that is not a real target can appear as noise or a false positive. Your connected-component step is intended to remove very small predicted clusters.

61. Vectorization

Converting a raster mask into geometric shapes such as polygons. Your project uses rasterio.features.shapes to convert the cleaned prediction into geographic geometry.

62. CRS (Coordinate Reference System)

A system defining how coordinates correspond to locations on Earth. Your pipeline preserves the image's CRS so predictions remain geographically meaningful.

63. Affine Transform

A mathematical mapping between raster pixel positions and geographic coordinates. Your pipeline uses the image's transform to map predicted pixels back to physical locations.

64. EPSG:4326

A geographic coordinate reference system based on latitude and longitude under WGS 84. Your final GeoJSON is described as using EPSG:4326 coordinates.

65. GeoJSON

A JSON format for storing geographic features such as points, lines, and polygons. Your project exports cleaned oil-spill regions as GeoJSON for map integration.

G. Programming & Engineering

66. Python

The programming language used for your project. It connects the deep-learning, image-processing, and geospatial libraries used throughout the pipeline.

67. PyTorch

A Python deep-learning framework used to build and train the neural network. Your project uses PyTorch with CUDA acceleration.

68. segmentation_models_pytorch (SMP)

A library providing segmentation architectures and related components. Your project uses it to construct the U-Net-based segmentation model.

69. NumPy

A Python library for numerical arrays and mathematical operations. In this type of pipeline it represents image data and numerical transformations as arrays.

70. OpenCV (cv2)

A computer-vision library containing image and connected-component operations. Your project uses it for morphological/noise-related post-processing and connected components.

71. Rasterio

A Python library for reading and writing geospatial raster data. Your project uses it to read TIFF imagery, preserve geospatial metadata, and create geographic shapes.

72. DataLoader

A PyTorch utility that supplies training data in batches. In your project it helps feed image/mask samples to the training process.

73. Algorithm / Pipeline

An algorithm is a defined computational procedure, while a pipeline is a sequence of processing stages. Your project is best understood as a machine-learning and geospatial processing pipeline rather than one single algorithm.

74. Problem-Solving Method

The project follows an applied ML workflow: define the segmentation problem, prepare labeled data, train a model, evaluate on held-out data, then post-process and georeference the result. Each stage transforms the output of the previous stage into something useful for the next.

75. Why Python for This Project?

Python provides the ecosystem connecting PyTorch, SMP, OpenCV, NumPy, and Rasterio. This lets one language handle model training, image processing, and geospatial output.

H. Likely Viva 'Why?' Questions

76. Why U-Net?

U-Net is designed for pixel-level segmentation and uses skip connections to retain spatial detail. Your project needs the location and shape of an oil region, not merely an image-level oil/no-oil answer.

77. Why ResNet34?

Your documentation says ResNet34 provides a balance of deep feature extraction and memory efficiency. This matters because your training hardware has an 8 GB RTX 4060.

78. Why two input channels?

The project input is Sentinel-1 dual-polarized VV/VH data, so two channels directly represent the available radar information. Using two channels avoids pretending the data is ordinary RGB imagery.

79. Why five classes?

The five classes separate sea, oil, look-alikes, ships, and land. This lets the model distinguish oil from several non-oil regions rather than treating every dark/anomalous region as oil.

80. Why Dice Loss?

The documentation says oil occupies a small fraction of ocean pixels, creating class imbalance. Dice directly emphasizes overlap between predicted and actual regions.

81. Why a small batch?

The U-Net's intermediate tensors and target encoding consume GPU memory. Your batch was reduced to 2 and sometimes 1 to avoid CUDA out-of-memory errors.

82. Why a held-out test set?

A final unseen set gives a cleaner measurement of generalization. Your documentation says 180 test images were isolated before training.

83. Why post-processing?

A neural network can produce small isolated predicted regions. Your pipeline uses connected-component analysis and a 50-pixel threshold to remove small clusters.

84. Why GeoJSON?

The model's raw output is a pixel mask, but a map system needs geographic geometry. GeoJSON provides a standard way to store the resulting geographic polygons.

I. What You Should Know Beyond the PDF

85. What if you change the batch size?

Increasing it processes more images simultaneously and generally requires more VRAM; decreasing it reduces memory usage but changes the training dynamics. In your project the hardware constraint forced the batch toward 2/1.

86. What if you train for more epochs?

More epochs give the model more opportunities to learn, but they do not automatically improve generalization. Validation behavior must be monitored to see whether additional training helps or starts to overfit.

87. What if test data enters training?

The final metric can become misleading because the model has indirectly seen information from the evaluation set. This is why your project isolates the test images.

88. What if oil and a look-alike appear similar?

The five-class setup gives the network a separate look-alike category instead of forcing every anomaly to become oil. The actual ability to distinguish them depends on the training data and learned features.

89. What if IoU is high but oil detection is poor?

Overall metrics can hide class-specific weaknesses when classes are imbalanced. That is why your documentation reports oil-specific Global IoU and Mean Dice in addition to mean image-wise IoU.

90. What is the full pipeline?

Sentinel-1 VV/VH TIFF → preprocessing → ResNet34 U-Net → five-class pixel prediction → Dice-based training → held-out evaluation → connected-component cleanup → geographic vectorization → EPSG:4326 GeoJSON.

Quick Reference: Your Project's Main Values

Item	Value / choice	What you should be able to explain
Language	Python	Why Python fits the ML + CV + GIS stack
Input	Sentinel-1 SAR	Why radar imagery is used
Channels	2: VV + VH	What each channel represents
Classes	5	Sea, Oil, Look-alike, Ship, Land
Architecture	U-Net + ResNet34	Encoder, decoder, skip/residual connections
Split	70 / 15 / 15	Train, validation, held-out test
Test set	180 images	Why it stays unseen
Batch	2, sometimes 1	8 GB VRAM constraint
Epochs	20	One full pass; value used in the documented run
Loss	Dice Loss	Why overlap helps with class imbalance
AMP	torch.amp.autocast	Lower precision for memory efficiency
Cleanup	50-pixel threshold	Remove small connected regions
Metrics	83.63 / 65.71 / 80.42%	Mean image-wise IoU / oil Global IoU / oil Mean Dice
Output	GeoJSON, EPSG:4326	Map-ready geographic polygons

Study priority: First understand what each stage does; then understand why your documented values were used; finally practice 'what if we changed it?' questions. You do not need to memorize the wording—understand the cause-and-effect chain.